

# Python 程式語言

## Lecture 7: Algorithm Implementation in Python

---

Instructor: 顏家珩 Chia-Heng Yen

---

# Course Roadmap

---

- Fundamentals & Environment
  - Introduction to Python and Development Environment Setup
  - Basic Syntax and Flow Control
  - Data Structures and Collections
- **Advanced Programming**
  - Modular and Functional Programming
  - File Handling and Exception Handling
  - Object-Oriented Programming
  - **Algorithm Implementation in Python**
  - Modules and Package Applications
- User Interface Development
  - GUI Programming with Tkinter and PyQt
- Data Processing & Analysis
  - Data Analysis Fundamentals
  - Scientific Computing Applications
  - Data Visualization
- AI & ML Applications
  - Introduction to Machine Learning with Applications
  - AI-Related Library in Python Applications

# Outline

---

- Implement and analyze fundamental sorting algorithms
- Apply searching algorithms effectively
- Master string matching with KMP algorithm
- Implement essential data structures
- Analyze algorithm efficiency with Big O notation

# Sorting Algorithms

---

# Sorting Algorithms

---

- Why Study Sorting?
- Importance
  - Foundation of Computer Science
  - Real-world Applications
  - Database indexing
  - Data analysis
  - Search optimization
- Algorithm Complexity Analysis

*# Small dataset*

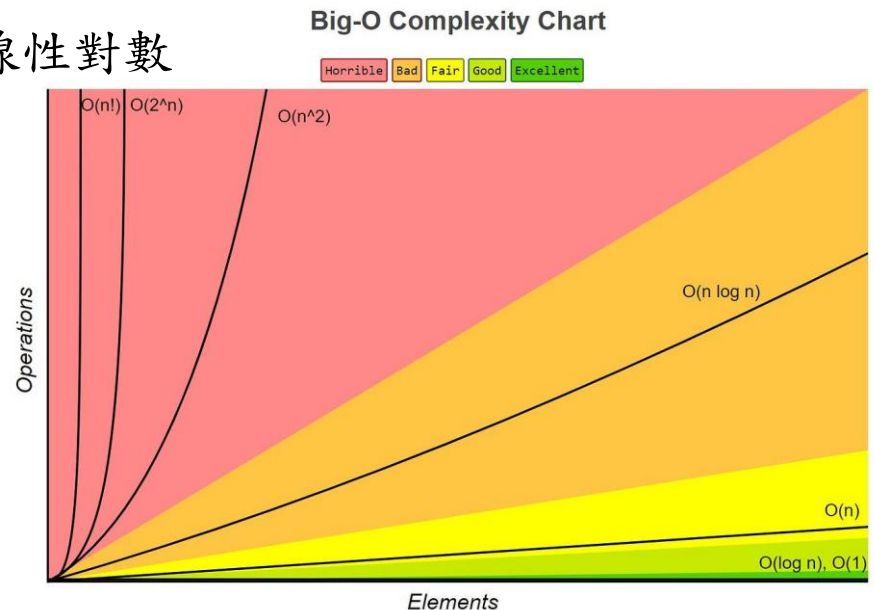
```
data = [3, 1, 4, 1, 5, 9, 2, 6] # Fast enough
```

*# Large dataset*

```
data = list(range(1000000, 0, -1)) # Need efficient algorithm!
```

# Algorithm Complexity

- Big O Notation
  - Time Complexity: How execution time grows with input size
  - Space Complexity: How memory usage grows with input size
- Common Complexity Classes
  - Time Complexity
    - $O(1)$  - Constant 常數
    - $O(\log n)$  - Logarithmic 對數
    - $O(n)$  - Linear 線性
    - $O(n \log n)$  - Linearithmic 線性對數
    - $O(n^2)$  - Quadratic 平方
    - $O(2^n)$  - Exponential 指數



# Bubble Sort

---

- Concept
  - Repeatedly swap adjacent elements if they are in wrong order
- Complexity
  - Time: Best  $O(n)$ , Average  $O(n^2)$ , Worst  $O(n^2)$
  - Space:  $O(1)$
  - Stable: Yes
- Visualization
  - Initial: [5, 3, 8, 4, 2]
  - Pass 1: [3, 5, 4, 2, 8] ← 8 bubbles to end
  - Pass 2: [3, 4, 2, 5, 8] ← 5 in position
  - Pass 3: [3, 2, 4, 5, 8] ← 4 in position
  - Pass 4: [2, 3, 4, 5, 8] ← Sorted!

# Bubble Sort Implementation

```
def bubble_sort(arr):

    n = len(arr)
    # Traverse through all array elements
    for i in range(n):
        # Flag to optimize
        swapped = False
        # Last i elements are already sorted
        for j in range(0, n - i - 1):
            # Swap if element is greater than next
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True

        # If no swap occurred, array is sorted
        if not swapped:
            break

    return arr
```

```
# Test
data = [64, 25, 12, 22, 11]
print(f"Original: {data}")
selection_sort(data)
print(f"Sorted: {data}")
```

# Selection Sort

---

- Concept
  - Find minimum element and place it at the beginning
- Complexity
  - Time:  $O(n^2)$  for all cases
  - Space:  $O(1)$
  - Stable: No
- Visualization
  - Initial: [64, 25, 12, 22, 11]
  - Step 1: [11, 25, 12, 22, 64] ← Find min (11), swap with first
  - Step 2: [11, 12, 25, 22, 64] ← Find min in rest (12)
  - Step 3: [11, 12, 22, 25, 64] ← Find min in rest (22)
  - Step 4: [11, 12, 22, 25, 64] ← Sorted!

# Selection Sort Implementation

```
def selection_sort(arr):  
  
    n = len(arr)  
    # Traverse through all array elements  
    for i in range(n):  
        # Find minimum element in unsorted  
        min_idx = i  
        for j in range(i + 1, n):  
            if arr[j] < arr[min_idx]:  
                min_idx = j  
        # Swap found minimum with first element  
        arr[i], arr[min_idx] = arr[min_idx], arr[i]  
  
    return arr
```

```
# Test  
data = [64, 25, 12, 22, 11]  
print(f"Original: {data}")  
selection_sort(data)  
print(f"Sorted: {data}")
```

# Insertion Sort

---

- Concept
  - Build sorted array one element at a time
- Complexity
  - Time: Best  $O(n)$ , Average  $O(n^2)$ , Worst  $O(n^2)$
  - Space:  $O(1)$
  - Stable: Yes
- Visualization
  - Initial: [64, 25, 12, 22, 11]
  - Step 1: [25, 64, 12, 22, 11] ← Pick 25, insert before 64
  - Step 2: [12, 25, 64, 22, 11] ← Pick 12, insert at beginning
  - Step 3: [12, 22, 25, 64, 11] ← Pick 22, insert between 12 and 25
  - Step 4: [11, 12, 22, 25, 64] ← Pick 11, insert at beginning, Sorted!

# Insertion Sort Implementation

```
def insertion_sort(arr):  
  
    for i in range(1, len(arr)):  
        key = arr[i]  
        j = i - 1  
  
        # Move elements greater than key one position ahead  
        while j >= 0 and arr[j] > key:  
            arr[j + 1] = arr[j]  
            j -= 1  
  
        # Insert key at correct position  
        arr[j + 1] = key  
    return arr
```

```
# Test  
data = [12, 11, 13, 5, 6]  
print(f"Original: {data}")  
insertion_sort(data)  
print(f"Sorted: {data}")
```

# Quick Sort

---

- Concept
  - Divide and conquer: pick pivot, partition, recursively sort
- Complexity
  - Time: Best  $O(n \log n)$ , Average  $O(n \log n)$ , Worst  $O(n^2)$
  - Space:  $O(\log n)$
  - Stable: No
- Visualization
  - Initial: [10, 7, 8, 9, 1, 5]
  - Choose pivot = 5
    - Partition: [1] (smaller) [5] (pivot) [7, 8, 9, 10] (larger)
  - Recursively sort left and right
  - Result: [1, 5, 7, 8, 9, 10]

# Merge Sort

---

- Concept
  - Divide array into halves, recursively sort, then merge
- Complexity
  - Time:  $O(n \log n)$  for all cases
  - Space:  $O(n)$
  - Stable: Yes
- Visualization
  - Initial: [38, 27, 43, 3, 9, 82, 10]
  - Divide:
    - [38, 27, 43, 3] [9, 82, 10]
    - [38, 27] [43, 3] [9, 82] [10]
    - [38] [27] [43] [3] [9] [82] [10]
  - Merge:
    - [27, 38] [3, 43] [9, 82] [10]
    - [3, 27, 38, 43] [9, 10, 82]
    - [3, 9, 10, 27, 38, 43, 82]

# Merge Sort Implementation

```
def merge_sort(arr):

    if len(arr) <= 1:
        return arr

    # Divide: find middle point
    mid = len(arr) // 2
    left = arr[:mid]
    right = arr[mid:]

    # Conquer: recursively sort both halves
    left = merge_sort(left)
    right = merge_sort(right)

    # Combine: merge sorted halves
    return merge(left, right)
```

```
def merge(left, right):
    """Merge two sorted arrays"""
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]) i += 1
        else:
            result.append(right[j]) j += 1

    # Append remaining elements
    result.extend(left[i:])
    result.extend(right[j:])

    return result

# Test
data = [38, 27, 43, 3, 9, 82, 10]
print(f"Original: {data}")
sorted_data = merge_sort(data)
print(f"Sorted: {sorted_data}")
```

# Sorting Algorithm Comparison

- Performance Summary

| Algorithm                    | Best          | Average       | Worst         | Space       | Stable | Use Case                 |
|------------------------------|---------------|---------------|---------------|-------------|--------|--------------------------|
| Bubble                       | $O(n)$        | $O(n^2)$      | $O(n^2)$      | $O(1)$      | Yes    | Educational only         |
| Selection                    | $O(n^2)$      | $O(n^2)$      | $O(n^2)$      | $O(1)$      | No     | Memory constrained       |
| Insertion                    | $O(n)$        | $O(n^2)$      | $O(n^2)$      | $O(1)$      | Yes    | Small/nearly sorted      |
| Quick                        | $O(n \log n)$ | $O(n \log n)$ | $O(n^2)$      | $O(\log n)$ | No     | General purpose          |
| Merge                        | $O(n \log n)$ | $O(n \log n)$ | $O(n \log n)$ | $O(n)$      | Yes    | Guaranteed $O(n \log n)$ |
| Timsort<br>(Python Built-in) | $O(n)$        | $O(n \log n)$ | $O(n \log n)$ | $O(n)$      | Yes    | Real-world data          |

# When to Use Which Sorting Algorithm?

---

- Use Insertion Sort when
  - Array size  $< 50$
  - Data is nearly sorted
  - Need simple, stable, in-place sort
- Use Merge Sort when
  - Need guaranteed  $O(n \log n)$
  - Stability is required
  - Have extra memory available
- Use Quick Sort when
  - Average case performance matters
  - Memory is limited
  - Data is random
- Use Python's `sort()`
  - General purpose sorting
  - Real-world data (often partially sorted)
  - Need best practical performance

# Timsort (Python Built-in Sorting)

- Python's built-in `sort()` and `sorted()` use Timsort, a hybrid algorithm combining **Merge Sort** and **Insertion Sort**
- Key Features

```
# Python's sort is highly optimized
```

```
# In-place sort
```

```
data = [3, 1, 4, 1, 5, 9, 2, 6]
data.sort()
print(data) # [1, 1, 2, 3, 4, 5, 6, 9]
```

```
# Returns new list
```

```
data = [3, 1, 4, 1, 5, 9, 2, 6]
sorted_data = sorted(data)
print(sorted_data) # [1, 1, 2, 3, 4, 5, 6, 9]
print(data) # [3, 1, 4, 1, 5, 9, 2, 6] (unchanged)
```

```
# Custom sorting with key
```

```
students = [
    {'name': 'Alice', 'grade': 85},
    {'name': 'Bob', 'grade': 92},
    {'name': 'Charlie', 'grade': 85}
]
```

```
# Sort by grade (stable - maintains order for equal values)
```

```
sorted_students = sorted(students, key=lambda x: x['grade'])
print(sorted_students)
```

```
# Reverse order
```

```
sorted_desc = sorted(students, key=lambda x: x['grade'], reverse=True)
print(sorted_desc)
```

# Why Timsort is default

---

- Adaptive -  $O(n)$  best case
- Stable - preserves order
- Optimized for real data
- Guaranteed  $O(n \log n)$  worst case

# Searching Algorithms

---

# Linear Search

---

- Concept
  - Check each element sequentially until target is found
- Complexity
  - Time:  $O(n)$
  - Space:  $O(1)$

# Linear Search Implementation

```
def linear_search(arr, target):
    """ Linear Search implementation, Checks each element one by one """
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1

# Find all occurrences
def linear_search_all(arr, target):
    """Find all positions where target appears"""
    return [i for i in range(len(arr)) if arr[i] == target]

# Test
numbers = [10, 23, 45, 70, 11, 15, 23]
print(f"Array: {numbers}")
print(f"Find 23: {linear_search(numbers, 23)}")
print(f"Find all 23: {linear_search_all(numbers, 23)}")
```

# Binary Search

---

- Concept
  - Divide and conquer on sorted array
- Complexity
  - Time:  $O(\log n)$
  - Space:  $O(1)$  iterative,  $O(\log n)$  recursive
  - Requirement: Array must be sorted

# Iterative Implementation of Binary Search

```
def binary_search(arr, target):  
    """ Binary Search - Iterative version, Requires sorted array """  
    left = 0  
    right = len(arr) - 1  
  
    while left <= right:  
        mid = (left + right) // 2  
        if arr[mid] == target:  
            return mid  
        elif arr[mid] < target:  
            left = mid + 1 # Search right half  
        else: right = mid - 1 # Search left half  
    return -1 # Not found  
  
# Test  
sorted_numbers = [1, 3, 5, 7, 9, 11, 13, 15, 17, 19]  
print(f"Sorted array: {sorted_numbers}")  
print(f"Find 7: {binary_search(sorted_numbers, 7)}")  
print(f"Find 10: {binary_search(sorted_numbers, 10)}")
```

# Recursive Implementation of Binary Search

```
def binary_search_recursive(arr, target, left, right):  
    """ Binary Search - Recursive version """  
    if left > right:  
        return -1  
  
    mid = (left + right) // 2  
  
    if arr[mid] == target:  
        return mid  
    elif arr[mid] < target:  
        return binary_search_recursive(arr, target, mid + 1, right)  
    else:  
        return binary_search_recursive(arr, target, left, mid - 1)  
  
# Test  
result = binary_search_recursive(sorted_numbers, 13, 0, len(sorted_numbers) - 1)  
print(f"Recursive find 13: {result}")
```

# Hash Table Search

---

- Concept
  - Use hash function to compute index for  $O(1)$  access
- Complexity
  - Time:  $O(1)$  average,  $O(n)$  worst
  - Space:  $O(n)$

# Python Dictionary as Hash Table

*# Python dict is implemented as hash table*

*# Create hash table*

```
phone_book = {  
    "Alice": "123-4567",  
    "Bob": "234-5678",  
    "Charlie": "345-6789"  
}
```

*# O(1) lookup*

```
print(f"Alice's phone: {phone_book['Alice']}")
```

*# O(1) check existence*

```
if "Bob" in phone_book:  
    print("Bob found")
```

*# O(1) insert*

```
phone_book["David"] = "456-7890"
```

*# O(1) delete*

```
del phone_book["Charlie"]  
print(f"Final phone book: {phone_book}")
```

# Search Algorithm Comparison

```
import time
import random

def compare_search_algorithms(size=10000):
    """ Compare search algorithm performance """
    # Create data
    data = sorted(random.sample(range(size * 10), size))
    target = random.choice(data)
    # Hash table
    hash_table = {val: i for i, val in enumerate(data)}
    print(f"Comparing search algorithms with {size} elements")
    # Linear Search
    start = time.time()
    linear_result = linear_search(data, target)
    linear_time = time.time() - start
    print(f"Linear Search: {linear_time:.6f}s")
```

```
# Binary Search
start = time.time()
binary_result = binary_search(data, target)
binary_time = time.time() - start
print(f"Binary Search: {binary_time:.6f}s")
# Hash Table
start = time.time()
hash_result = hash_table.get(target, -1)
hash_time = time.time() - start
print(f"Hash Table: {hash_time:.6f}s")
print(f"\nSpeedup vs Linear:")
print(f" Binary: {linear_time/binary_time:.0f}x faster")
print(f" Hash: {linear_time/hash_time:.0f}x faster")

compare_search_algorithms(10000)
```

Comparing search algorithms with 10000 elements

Linear Search: 0.000427s

Binary Search: 0.000005s

Hash Table: 0.000001s

Speedup vs Linear:

Binary: 85x faster

Hash: 359x faster

# String Matching Algorithms

---

# Introduction to String Matching

---

- The Problem
  - Given
    - Text  $T$  of length  $n$  (the haystack)
    - Pattern  $P$  of length  $m$  (the needle)
  - Find
    - All occurrences of  $P$  in  $T$
- Example
  - Text: "ABABDABACDABABCABAB"
  - Pattern: "ABABCABAB"
  - Output: [10] (0-indexed)

# Real-World Applications

---

- Text Editors: Find & replace
- Search Engines: Query matching
- DNA Analysis: Gene sequence matching
- Plagiarism Detection: Document comparison
- Network Security: Intrusion detection

# Naive String Matching

- Brute Force Approach

```
def naive_string_match(text, pattern):  
    """ Naive string matching algorithm, Time  
    Complexity: O(n*m), Space Complexity: O(1) """  
    n = len(text)  
    m = len(pattern)  
    positions = []  
  
    # Try every possible starting position  
    for i in range(n - m + 1):  
        # Check if pattern matches at position i  
        match = True  
        for j in range(m):  
            if text[i + j] != pattern[j]:  
                match = False  
                break  
        if match:  
            positions.append(i)  
    return positions
```

```
# Test  
text = "ABABDABACDABABCABAB"  
pattern = "ABABCABAB"  
positions = naive_string_match(text, pattern)  
print(f"Text: {text}")  
print(f"Pattern: {pattern}")  
print(f"Found at positions: {positions}")
```

**Key Problem:** Naive algorithm re-compares characters unnecessarily. It doesn't learn from previous comparisons.

# KMP (Knuth-Morris-Pratt) Algorithm

---

- The Key Insight
  - When a mismatch occurs, the pattern itself contains information about where the next match could begin.
- Example
  - Text: A B A B D A B A C D A B A B C A B A B
  - Pattern: A B A B C A B A B
  - When we mismatch at position 4 ( $D \neq C$ ):
    - We already know: "ABAB" matched
    - Rather than starting over, we can skip ahead because "AB" at the beginning = "AB" at the end of what matched

# Understanding the LPS Array

---

- What is LPS?
  - LPS = Longest Proper Prefix which is also Suffix
    - Proper prefix: prefix that is not the entire string
    - Suffix: ending substring
- Visual Examples
  - Pattern: ABABCABAB
  - Index: 0 1 2 3 4 5 6 7 8
  - LPS: 0 0 1 2 0 1 2 3 4

# Visual Examples

- Pattern: A B A B C A B A B
- Index: 0 1 2 3 4 5 6 7 8
- LPS: 0 0 1 2 0 1 2 3 4
- Explanation
  - Index 0: "A"
    - No proper prefix
    - LPS[0] = 0
  - Index 1: "AB"
    - Prefixes: "A"
    - Suffixes: "B"
    - No match, LPS[1] = 0
  - Index 2: "ABA"
    - Prefixes: "A", "AB"
    - Suffixes: "A", "BA"
    - Match: "A" = "A" (length 1), LPS[2] = 1
  - Index 3: "ABAB"
    - Prefixes: "A", "AB", "ABA"
    - Suffixes: "B", "AB", "BAB"
    - Match: "AB" = "AB" (length 2), LPS[3] = 2
  - ...

# KMP Search Algorithm

---

- Step 1: Build LPS Array
- Step 2: Search Process
- Text:    A B A B D A B A C D A B A B C A B A B
- Pattern: A B A B C A B A B
- LPS:    0 0 1 2 0 1 2 3 4

# Search Process

---

- Iteration 1-4: First 4 characters match
  - $i=0, j=0$ :
  - Text:    A B A B D A B A C D A B A B C A B A B
  - Pattern: A B A B C A B A B
  - Match!  $\rightarrow i=1, j=1$
  - ...
  - $i=3, j=3$ :
  - Text:    A B A B D A B A C D A B A B C A B A B
  - Pattern: A B A B C A B A B
  - Match!  $\rightarrow i=4, j=4$

# Search Process

- Iteration 5: Mismatch!
  - $i=4, j=4$ :
  - Text:     A B A B D A B A C D A B A B C A B A B
  - Pattern:   A B A B C A B A B
  - Mismatch!  $\rightarrow D \neq C$
- What does naive algorithm do?
  - $\rightarrow$  Start over from position 1
- What does KMP do?
  - $\rightarrow$  Look at  $LPS[j-1] = LPS[3] = 2$
  - $\rightarrow$  Set  $j = 2$  (skip unnecessary comparisons!)
- $i=4, j=2$ :
- Text:         A B A B D A B A C D A B A B C A B A B
- Pattern:       A B A B C A B A B
- Mismatch!  $\rightarrow D \neq A$
- ...

# Key Advantages

---

- Linear Time:  $O(n + m)$  guaranteed
- No Backtracking: Text pointer never goes backward
- Predictable Performance: No worst case like  $O(n \times m)$
- Efficient for Patterns: Especially those with repeated substrings

# Data Structure Implementation

---

# Stack

---

- Concept
  - LIFO - Last In, First Out
- Operations
  - Push: Add to top -  $O(1)$
  - Pop: Remove from top -  $O(1)$
  - Peek: View top -  $O(1)$

# Stack Implementation

```
class Stack:
    """ Stack implementation using Python list"""

    def __init__(self):
        self.items = []

    def is_empty(self):
        """Check if stack is empty"""
        return len(self.items) == 0

    def push(self, item):
        """Add item to top"""
        self.items.append(item)

    def pop(self):
        """Remove and return top item"""
        if self.is_empty():
            raise IndexError("Pop from empty stack")
        return self.items.pop()
```

```
    def peek(self):
        """View top item"""
        if self.is_empty():
            raise IndexError("Peek from empty stack")
        return self.items[-1]

    def size(self):
        """Return number of items"""
        return len(self.items)

# Test
stack = Stack()
stack.push(1)
stack.push(2)
stack.push(3)
print(f"Top: {stack.peek()}")    # 3
print(f"Pop: {stack.pop()}")    # 3
print(f"Size: {stack.size()}")  # 2
```

# Stack Applications

## # 1. Balanced Parentheses

```
def is_balanced(expression):
    """Check if parentheses are balanced"""
    stack = []
    pairs = {'(': ')', '[': ']', '{': '}' }

    for char in expression:
        if char in pairs:
            stack.append(char)
        elif char in pairs.values():
            if not stack or pairs[stack.pop()] != char:
                return False

    return len(stack) == 0

print(is_balanced("((()))"))      # True
print(is_balanced("({[]})"))      # True
print(is_balanced("(O)"))         # False
```

## # 2. Reverse String

```
def reverse_string(s):
    """Reverse string using stack"""
    stack = Stack()

    for char in s:
        stack.push(char)

    result = ""
    while not stack.is_empty():
        result += stack.pop()

    return result

print(reverse_string("Python")) # nohtyP
```

# Queue

---

- Concept
  - FIFO - First In, First Out
- Operations
  - Enqueue: Add to rear -  $O(1)$
  - Dequeue: Remove from front -  $O(1)$
  - Front: View front element -  $O(1)$

# Queue Implementation

```
from collections import deque

class Queue:
    """Queue implementation using deque"""
    def __init__(self):
        self.items = deque()
    def is_empty(self):
        """Check if queue is empty"""
        return len(self.items) == 0
    def enqueue(self, item):
        """Add item to rear"""
        self.items.append(item)
    def dequeue(self):
        """Remove and return front item"""
        if self.is_empty():
            raise IndexError("Dequeue from empty queue")
        return self.items.popleft()
```

```
    def front(self):
        """View front item"""
        if self.is_empty():
            raise IndexError("Front from empty queue")
        return self.items[0]
    def size(self):
        """Return number of items"""
        return len(self.items)

# Test
queue = Queue()
queue.enqueue("First")
queue.enqueue("Second")
queue.enqueue("Third")
print(f"Front: {queue.front()}") # First
print(f"Dequeue: {queue.dequeue()}") # First
print(f"Size: {queue.size()}") # 2
```

# Linked List

---

- Concept
  - Linear data structure where elements are stored in nodes
- Node Structure
  - Data: stores value
  - Next: points to next node

# Linked List Implementation

```
class Node:
    """Node for linked list"""
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    """ Singly linked list implementation """
    def __init__(self):
        self.head = None
    def is_empty(self):
        """Check if list is empty"""
        return self.head is None
    def insert_at_beginning(self, data):
        """Insert at beginning - O(1)"""
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node
```

```
    def insert_at_end(self, data):
        """Insert at end - O(n)"""
        new_node = Node(data)
        if self.is_empty():
            self.head = new_node
            return
        current = self.head
        while current.next:
            current = current.next
        current.next = new_node

    def delete(self, data):
        """Delete first occurrence"""
        if self.is_empty():
            return
        if self.head.data == data:
            self.head = self.head.next
            return
```

# Linked List Implementation

```
current = self.head
while current.next and current.next.data != data:
    current = current.next
if current.next:
    current.next = current.next.next
def display(self):
    """Display list"""
    if self.is_empty():
        print("Empty list")
        return
    elements = []
    current = self.head
    while current:
        elements.append(str(current.data))
        current = current.next
    print(" → ".join(elements))
```

```
# Test
ll = LinkedList()
ll.insert_at_end(1)
ll.insert_at_end(2)
ll.insert_at_end(3)
ll.display() # 1 → 2 → 3
ll.delete(2)
ll.display() # 1 → 3
```

# Binary Tree

---

- Concept
  - Hierarchical data structure with nodes having at most two children

# Binary Tree Implementation

```
class TreeNode:
    """Node for binary tree"""
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

class BinaryTree:
    """ Binary tree with traversals"""
    def __init__(self, root_data=None):
        self.root = TreeNode(root_data) if
root_data else None
    def inorder(self, node):
        """Inorder: Left → Root → Right"""
        if node:
            self.inorder(node.left)
            print(node.data, end=" ")
            self.inorder(node.right)
```

```
    def preorder(self, node):
        """Preorder: Root → Left → Right"""
        if node:
            print(node.data, end=" ")
            self.preorder(node.left)
            self.preorder(node.right)
    def postorder(self, node):
        """Postorder: Left → Right → Root"""
        if node:
            self.postorder(node.left)
            self.postorder(node.right)
            print(node.data, end=" ")

tree = BinaryTree(1)
tree.root.left = TreeNode(2)
tree.root.right = TreeNode(3)
tree.root.left.left = TreeNode(4)
tree.root.left.right = TreeNode(5)
```

```
print("Inorder:", end=" ")
tree.inorder(tree.root)
print()

print("Preorder:", end=" ")
tree.preorder(tree.root)
print()

print("Postorder:", end=" ")
tree.postorder(tree.root)
print()

# Create tree
#      1
#     / \
#    2   3
#   / \
#  4   5
```